

The operational stuff people forget

Running a vector database in production has some specific considerations that don't always show up in tutorials.

Memory matters a lot: Index structures like HNSW are typically held in RAM for fast access. If you have ten million 768-dimensional vectors, you're looking at tens of gigabytes of RAM just for the index. Some databases offer disk-based options that trade speed for cost, but plan for the RAM bill if you want fast search.

Backups and restores aren't always as smooth as in mature databases: Vector databases are newer software. Test your backup and restore procedures before you need them.

Re-indexing is sometimes required: If you change your embedding model, if your data drifts significantly, and if you change index parameters, you may need to rebuild your indexes. This is fine if you plan for it. Painful if you don't.

Monitoring is a bit different: Beyond the usual database metrics, you want to watch query latency, recall rates if you can measure them, and the distribution of vectors over time. Most vector databases now expose Prometheus-compatible metrics, which makes life easier.

Multi-tenancy gets tricky: If you're building a SaaS where each customer has their own data, you need to decide whether each gets their own collection, you partition a shared collection, or you store everything together and filter aggressively. Each approach has trade-offs around performance, isolation, and cost.

What you can do this week

If you're new to vector databases, here's a path I'd suggest for getting hands-on.

This weekend, install Chroma or Qdrant locally. Pick a small dataset you understand -- maybe a few hundred Wikipedia articles or your company's docs. Embed them with sentence-transformers. Write a tiny script that does similarity search. Play with it for an hour.

Next week, build something tiny but real. A search interface for your own notes. A 'find similar' feature for your blog. A semantic FAQ for a side project. The point isn't the product. The point is how the pieces fit together.

This month, learn about hybrid search and how to combine vector and keyword results. Read about how chunking strategy affects retrieval quality. Try a different embedding model and compare results. Get a sense of how much these choices matter in practice.

Beyond that, the path naturally branches into whatever direction your work pulls you. RAG pipelines if you're building AI features. Recommendation systems if you're in e-commerce or media. Internal search tools if you're at a larger company with lots of documentation.

Where is this all heading

A few trends worth keeping an eye on.

Hybrid search is becoming standard: Most vector databases now have first-class support for combining vector and keyword results, often using algorithms like Reciprocal Rank Fusion. Pure vector search is no longer the default approach.

Embeddings are getting smaller and better: Models like Matryoshka embeddings let you generate vectors that can be truncated to smaller sizes without losing too much quality. This is great for storage and speed.

Multimodal search is becoming practical: Models that produce embeddings spanning text, images, and audio in the same space are improving fast. Searching for an image with a text query, or finding audio matching a description, is becoming routine.

Specialised hardware is starting to matter: Some vector databases are starting to use GPUs or specialised search hardware to accelerate similarity computation. For very large scale workloads, this could change the cost structure significantly.

The lines between vector databases and traditional databases keep blurring. Postgres has pgvector. MongoDB has Atlas Vector Search. Elasticsearch has dense vector fields. Redis has vector similarity. The question increasingly isn't "Should I use a vector database?" but "Which of my existing systems is the best place to store these vectors?"

Quiet plumbing that powers loud AI

Vector databases are perhaps the least glamorous infrastructure in the AI boom. Nobody writes blog posts about them the way they write about new language models. No founder gives a keynote about the genius of their indexing algorithm. But every AI app you've used in the last year -- the chatbots, the search tools, the recommendation engines, the document assistants -- has one of these things humming away underneath it, quietly making the rest possible.

Getting comfortable with vector databases right now is a bit like getting comfortable with relational databases in the late nineties. The technology will get cheaper, easier, and more invisible. But the people who really understand what's happening have an edge. Pick one, install it tonight, build something silly with it tomorrow. Six months from now, when somebody on your team asks why you're using Qdrant instead of MySQL, you'll have a good answer ready. And probably a coffee in your hand. **END** 🐧

By: Vishal Tanna

The author is the CEO of TechCrave, a technology services company that helps enterprises adopt open source, cloud native, and AI-driven solutions. He has over 15 years of experience in software development and technology leadership, and is particularly interested in scalable architectures and digital transformation.